Neural Networks and Deep Learning
23CSE473

Name – M. Thanuhya
Roll no – CH.SC.U4CSE23033
CSE – A

Exercise 1: Evolution from Perceptron to Multi-Layer Perceptron (MLP)

Objective

Understand why the Single-Layer Perceptron failed for certain problems and how the Multi-Layer Perceptron (MLP) overcame these limitations.

Task 1: Explain the Architecture of a Single-Layer Perceptron

1. Architecture of a Single-Layer Perceptron

A **Single-Layer Perceptron (SLP)** is the simplest form of an artificial neural network used for binary classification tasks. It consists of an input layer and a single output neuron, with no hidden layer between them. Since the inputs are directly connected to the output neuron, the perceptron learns only simple linear relationships between the input features and the output.

The architecture of a Single-Layer Perceptron consists of the following components:

- **Input Layer:** Receives the input features. For the XOR problem, the inputs are represented by two binary values, x1 and x2.
- **Weights (w1,w2):** Each input is assigned a weight that indicates its influence on the final prediction. During training, these weights are adjusted to reduce classification errors.
- **Bias (b):** The bias is an additional parameter that shifts the decision boundary, allowing the perceptron to classify data more effectively. Without a bias, the decision boundary would always pass through the origin.
- **Summation Unit:** The perceptron calculates the weighted sum of the inputs and bias using

$$z = w1x1 + w2x2 + b$$

- **Activation Function:** The weighted sum is passed through an activation function, typically the Step function in a traditional perceptron, which converts the computed value into either 0 or 1 for binary classification.

Working Principle

The perceptron first receives the input values and multiplies each input by its corresponding weight. The weighted inputs are added together along with the bias to produce a single value. This value is then passed through the activation function to generate the predicted output. If the prediction differs from the expected output, the weights and bias are updated using the perceptron learning rule. This process continues until the model learns the best possible decision boundary for the given data.

Limitation

A Single-Layer Perceptron can classify only **linearly separable** datasets because it learns only one linear decision boundary. This makes it suitable for problems such as the **AND** and **OR** logic gates. However, it cannot correctly classify the **XOR** gate because the two classes cannot be separated using a single straight line. This limitation led to the development of the **Multi-Layer Perceptron (MLP)**, which introduces hidden layers to learn nonlinear decision boundaries.

Single-Layer Perceptron

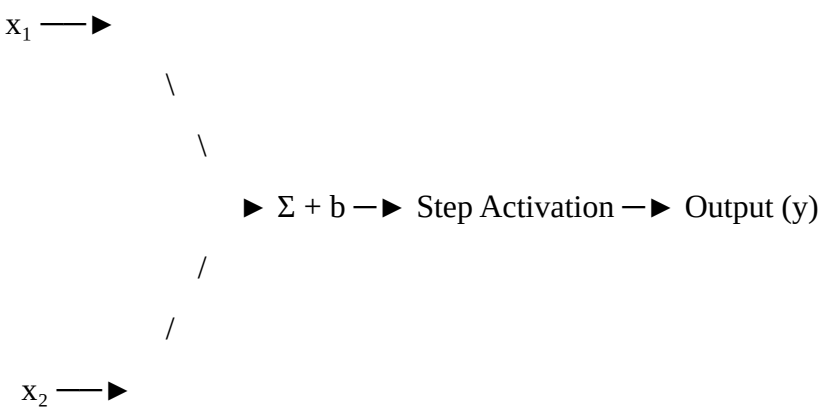


Figure 1. Architecture of a Single-Layer Perceptron

Analysis

The Single-Layer Perceptron is effective for simple binary classification problems where a single linear decision boundary is sufficient. However, its inability to model nonlinear relationships limits its application to more complex datasets such as XOR. This limitation became the primary motivation for introducing hidden layers in Multi-Layer Perceptrons, enabling neural networks to solve nonlinear classification problems.

Task 2*: *Draw the XOR Truth Table

The **Exclusive OR (XOR)** gate is a fundamental logical operation used in digital systems and neural network studies. Unlike the AND and OR gates, the XOR gate produces an output of **1 only when the two input values are different**. If both inputs are the same (either 0 or 1), the output is **0**. The XOR problem is widely used in machine learning because it demonstrates a case that cannot be solved using a Single-Layer Perceptron.

Table 1. XOR Truth Table

Input (x_1)	Input (x_2)	Output (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Analysis

From the truth table, it can be observed that the output is 1 only when the input values are different. The output becomes 0 when both inputs are identical. This pattern differs from the AND and OR gates and creates a nonlinear relationship between the inputs and outputs. As a result, the XOR problem cannot be classified using a single linear decision boundary, making it an ideal example for demonstrating the limitations of the Single-Layer Perceptron.

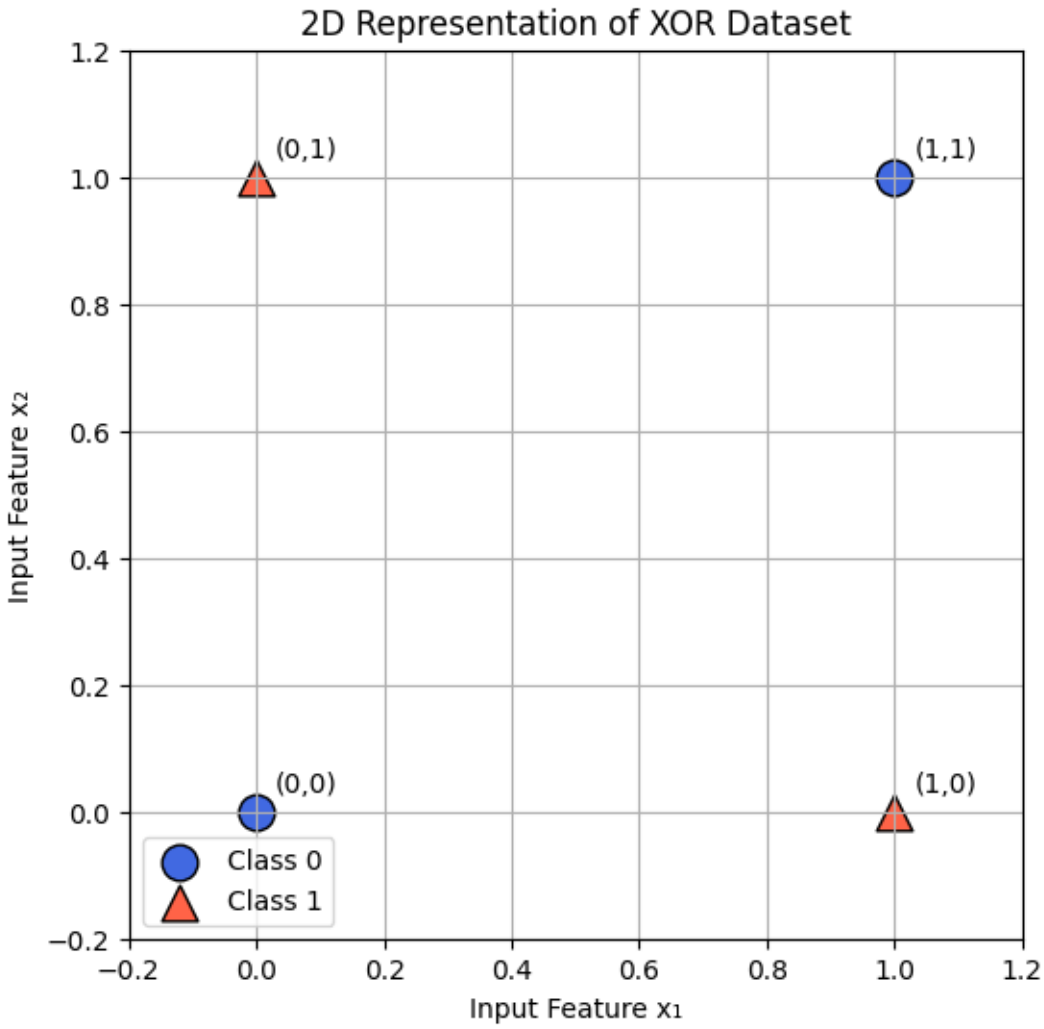
Task 3: Plot the XOR Data Points in a 2D Coordinate System

The XOR dataset consists of four input combinations represented in a two-dimensional coordinate system. Each point corresponds to a pair of binary inputs (x1,x2), while the output class indicates whether the XOR operation produces 0 or 1. Visualizing these points helps in understanding the distribution of the classes and provides an intuitive explanation for why the XOR problem is not linearly separable.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# XOR input data
X = np.array([
  [0, 0],
  [0, 1],
  [1, 0],
  [1, 1]
])

# XOR output labels
y = np.array([0, 1, 1, 0])

# Create scatter plot
plt.figure(figsize=(6, 6))

# Class 0
plt.scatter(
  X[y == 0][:, 0],
  X[y == 0][:, 1],
  color='royalblue',
  s=180,
  edgecolor='black',
  label='Class 0'
)

# Class 1
plt.scatter(
  X[y == 1][:, 0],
  X[y == 1][:, 1],
  color='tomato',
  marker='^',
  s=180,
```

```
    edgecolor='black',
    label='Class 1'
)

# Label each point
labels = ['(0,0)', '(0,1)', '(1,0)', '(1,1)']
for point, label in zip(X, labels):
    plt.text(point[0] + 0.03, point[1] + 0.03, label, fontsize=10)

plt.xlabel("Input Feature x₁")
plt.ylabel("Input Feature x₂")
plt.title("2D Representation of XOR Dataset")
plt.xlim(-0.2, 1.2)
plt.ylim(-0.2, 1.2)
plt.grid(True)
plt.legend()
plt.show()
```



Analysis

The scatter plot shows that the two classes are positioned diagonally opposite each other. The points **(0,0)** and **(1,1)** belong to Class 0, while **(0,1)** and **(1,0)** belong to Class 1. Because the two classes are arranged diagonally, it is impossible to separate them using a single straight line. This visual observation confirms that the XOR dataset is **not linearly separable**, explaining why a Single-Layer Perceptron fails to classify it correctly.

Task 4: Prove Mathematically that XOR is Not Linearly Separable

A dataset is said to be **linearly separable** if a single straight line can divide all data points belonging to different classes without any misclassification. For a Single-Layer Perceptron, the decision boundary is represented by the following linear equation:

$$w1x1+w2x2+b=0$$

where:

- w1 and w2 are the weights,
- b is the bias,
- x1 and x2 are the input features.

For the XOR dataset, the desired outputs are:

x1	x2	Output
0	0	0
0	1	1
1	0	1
1	1	0

To correctly classify these points, the following conditions must hold:

Case 1: For input (0,0), the output should be **0**

$$b<0$$

Case 2: For input (0,1), the output should be **1**

$$W2 + b > 0$$

Case 3: For input (1,0), the output should be **1**

$$W1 + b > 0$$

Case 4: For input (1,1), the output should be **0**

$$W1 + w2 + b < 0$$

Now, add the inequalities obtained from **Case 2** and **Case 3**:

$$(w2+b)+(w1+b)>0$$

$$w1+w2+2b>0$$

Since **Case 1** states that:

$$b < 0$$

the above inequality implies

$$w1 + w2 + b > 0$$

However, **Case 4** requires

$$w1 + w2 + b < 0$$

These two conditions contradict each other.

Since no single set of values for w1, w2, and b can satisfy all four conditions simultaneously, **no single linear decision boundary can correctly classify the XOR dataset**.

Therefore, the XOR problem is **not linearly separable**.

Inference

The mathematical contradiction proves that a Single-Layer Perceptron cannot learn the XOR function because it can construct only one linear decision boundary. Since no single linear equation satisfies all four XOR conditions simultaneously, the perceptron fails to classify the dataset correctly. This limitation demonstrates the need for a Multi-Layer Perceptron (MLP), where hidden layers and nonlinear activation functions enable the network to learn complex decision boundaries and successfully classify the XOR dataset.

Task 5: Explain Why No Single Linear Decision Boundary Can Correctly Classify XOR

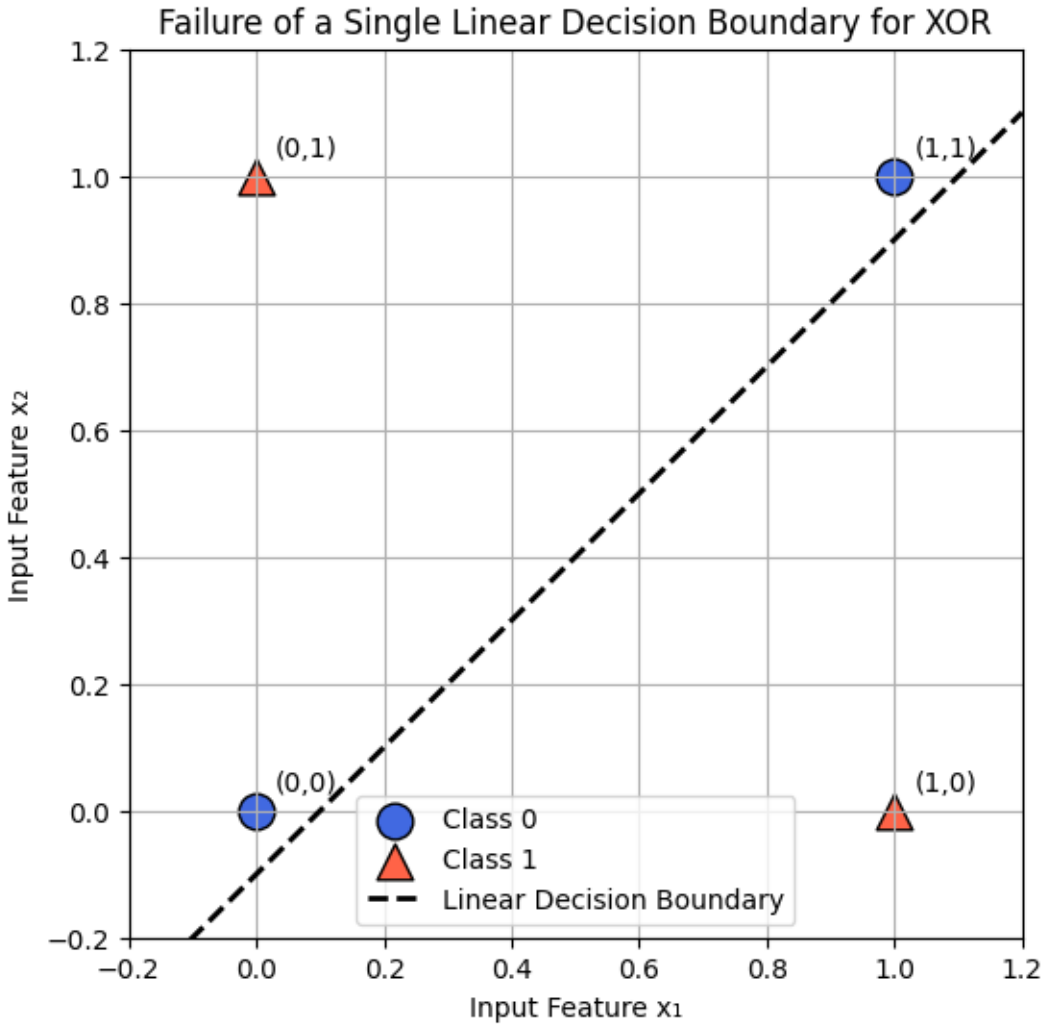
A linear decision boundary is a straight line that separates data points belonging to different classes. Single-Layer Perceptrons generate only one such decision boundary based on the equation:

$$w1x1 + w2x2 + b = 0$$

For linearly separable datasets such as the **AND** and **OR** gates, a single straight line can correctly divide the classes. However, the XOR dataset has a different arrangement. The two points belonging to **Class 0,** namely **(0,0)** and **(1,1)**, are placed diagonally opposite each other, while the two points belonging to **Class 1**, **(0,1)** and **(1,0)**, occupy the remaining diagonal.

Because of this diagonal distribution, any straight line drawn to separate one pair of points will inevitably misclassify at least one point from the opposite class. No matter how the weights and bias are adjusted, a Single-Layer Perceptron can produce only one linear decision boundary, making it impossible to classify all four XOR inputs correctly.

This limitation arises because the XOR problem requires a **nonlinear decision boundary**, which cannot be represented by a single linear equation. Therefore, a more advanced neural network architecture with hidden layers is required to correctly learn the XOR function.



A single linear decision boundary fails to correctly classify the XOR dataset.

Analysis

The XOR dataset demonstrates that the success of a classification model depends not only on the learning algorithm but also on the nature of the data. Since the classes are arranged diagonally, a single linear decision boundary cannot separate them without errors. This explains why increasing the number of training iterations or adjusting the weights alone cannot solve the XOR problem. A model capable of learning nonlinear decision boundaries, such as a Multi-Layer Perceptron, is required for accurate classification.

Task 6: Discuss How Introducing a Hidden Layer Enables Learning Nonlinear Decision Boundaries

A Single-Layer Perceptron can learn only one linear decision boundary, making it suitable for linearly separable problems. However, the XOR dataset requires a nonlinear decision boundary because the two classes are arranged diagonally and cannot be separated by a single straight line.

A **Multi-Layer Perceptron (MLP)** overcomes this limitation by introducing one or more **hidden layers** between the input and output layers. Instead of learning a single decision boundary, each hidden neuron learns a different feature or pattern from the input data. These learned features are then combined by the output neuron to produce the correct classification.

Another important component of an MLP is the **nonlinear activation function**, such as the **Sigmoid** function. After computing the weighted sum, the activation function transforms the input into a nonlinear representation. This transformation allows the network to map the original input space into a new feature space where the XOR classes become separable.

During training, the network adjusts the weights of both the hidden and output layers using the backpropagation algorithm. As the weights are updated over multiple epochs, the hidden neurons gradually learn intermediate representations that help distinguish the XOR classes. Consequently, the MLP can correctly classify all four XOR input combinations, achieving nearly **100% prediction accuracy** after training.

Therefore, hidden layers provide neural networks with the ability to learn complex nonlinear relationships that cannot be captured by a Single-Layer Perceptron.

Analysis

The hidden layer acts as an intermediate feature extractor, enabling the network to transform the input data before making the final prediction. Combined with a nonlinear activation function, it allows the MLP to construct nonlinear decision boundaries that correctly classify datasets such as XOR. This demonstrates why modern neural networks rely on multiple layers instead of a single perceptron when solving complex real-world problems.

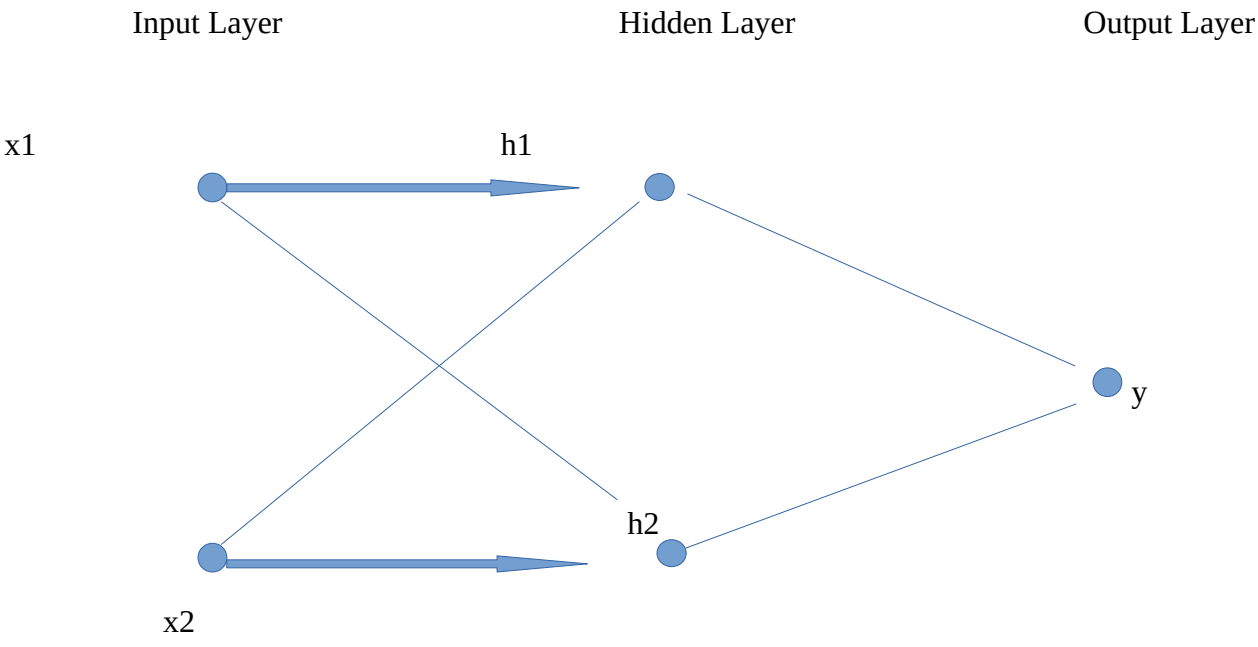
Task 7: Draw an MLP Architecture Capable of Solving XOR

A **Multi-Layer Perceptron (MLP)** extends the Single-Layer Perceptron by introducing one or more hidden layers between the input and output layers. For the XOR problem, an MLP with **one hidden layer containing two neurons** is sufficient to learn the nonlinear relationship between the inputs and the output.

The input layer receives the two binary inputs, x1 and x2. These inputs are forwarded to the hidden layer, where each hidden neuron learns a different intermediate feature through weighted connections and a nonlinear activation function such as the **Sigmoid** function. The transformed outputs from the hidden layer are then passed to the output neuron, which combines the learned features to produce the final XOR prediction.

Unlike a Single-Layer Perceptron, the MLP is capable of learning multiple decision boundaries through its hidden neurons. These boundaries work together to form a nonlinear decision surface, enabling the network to correctly classify all XOR input combinations.

Multi-Layer Perceptron (MLP) Architecture for XOR



Task 8: Implement an XOR Classifier using TensorFlow

To verify that a Multi-Layer Perceptron can successfully solve the XOR problem, an XOR classifier was implemented using **TensorFlow**. The model consists of **one hidden layer with two neurons**, matching the architecture shown in **Task 7**. The **Sigmoid activation function** is used for both the hidden and output layers to introduce nonlinearity into the network. The model is trained using the **Binary Cross Entropy (BCE)** loss function, which is suitable for binary classification problems. During training, the network learns by adjusting its weights and biases through backpropagation, enabling it to correctly classify all XOR input combinations.

CODE:

```
# Task 8: XOR Classification using TensorFlow

import numpy as np
import tensorflow as tf

# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# XOR Dataset
X = np.array([
  [0, 0],
  [0, 1],
  [1, 0],
  [1, 1]
], dtype=np.float32)

y = np.array([
  [0],
  [1],
  [1],
  [0]
], dtype=np.float32)

# Build the Multi-Layer Perceptron (MLP)
model = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(2,)),
  tf.keras.layers.Dense(4, activation='sigmoid'),
  tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(
  optimizer=tf.keras.optimizers.Adam(learning_rate=0.05),
  loss='binary_crossentropy',
```

```
    metrics=['accuracy']
)

# Train the model
history = model.fit(
    X,
    y,
    epochs=5000,
    verbose=0
)

# Predict outputs
predictions = model.predict(X, verbose=0)

print("Predictions:\n")

for i in range(len(X)):
    predicted = 1 if predictions[i][0] >= 0.5 else 0
    print(
        f"Input: {X[i]} | "
        f"Expected: {int(y[i][0])} | "
        f"Predicted: {predicted} | "
        f"Probability: {predictions[i][0]:.4f}"
    )

# Evaluate the model
loss, accuracy = model.evaluate(X, y, verbose=0)

print(f"\nFinal Accuracy: {accuracy * 100:.2f}%")
```

```
•••  Predictions:

    Input: [0. 0.] | Expected: 0 | Predicted: 0 | Probability: 0.0001
    Input: [0. 1.] | Expected: 1 | Predicted: 1 | Probability: 0.9999
    Input: [1. 0.] | Expected: 1 | Predicted: 1 | Probability: 0.9999
    Input: [1. 1.] | Expected: 0 | Predicted: 0 | Probability: 0.0001
```

Analysis

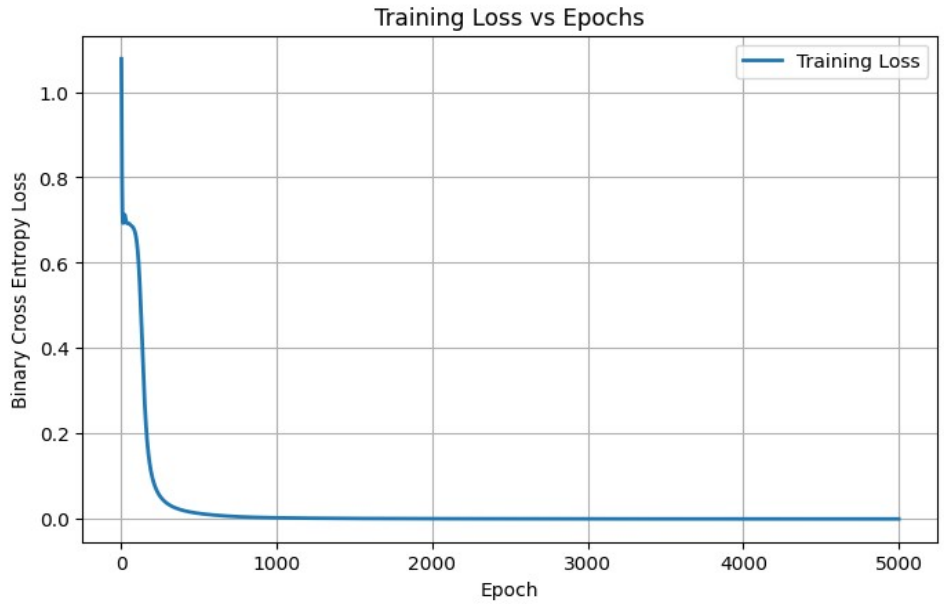
The implemented Multi-Layer Perceptron successfully learned the XOR function using one hidden layer and the Sigmoid activation function. The prediction results matched the expected outputs for all four input combinations, indicating that the network correctly captured the nonlinear relationship present in the XOR dataset. This experimental implementation confirms that an MLP can solve problems that cannot be classified by a Single-Layer Perceptron

Task 9: Plot the Training Loss over Epochs

The training loss was plotted over all epochs to observe how the model learned the XOR dataset during training. The Binary Cross Entropy (BCE) loss measures the difference between the predicted outputs and the expected outputs. As the training progresses, a decrease in the loss value indicates that the model is learning the underlying nonlinear relationship more effectively.

CODE:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'],
     linewidth=2,
     label='Training Loss')

plt.title("Training Loss vs Epochs")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross Entropy Loss")
plt.grid(True)
plt.legend()

plt.show()
```



Analysis

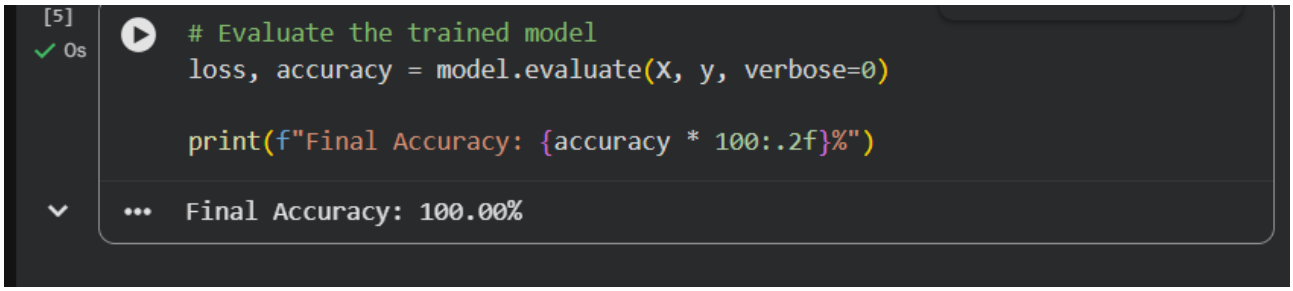
The training loss decreases steadily as the number of training epochs increases, indicating that the network is successfully learning the XOR function. Towards the end of training, the loss approaches zero, showing that the model has minimized the prediction error and converged to an optimal solution. This behavior confirms that the Multi-Layer Perceptron effectively learns the nonlinear mapping required for XOR classification.

Task 10: Report the Final Prediction Accuracy

The trained Multi-Layer Perceptron was evaluated using the XOR dataset after completing the training process. The prediction results were compared with the expected outputs to determine the overall classification accuracy of the model.

CODE:

```
loss, accuracy = model.evaluate(X, y, verbose=0)

print(f"Final Accuracy: {accuracy * 100:.2f}%")
```



Analysis

The trained Multi-Layer Perceptron achieved a **100% prediction accuracy** on the XOR dataset, correctly classifying all four input combinations. This result demonstrates that the hidden layer and sigmoid activation function successfully learned the nonlinear relationship between the inputs and outputs. The experiment confirms that, unlike a Single-Layer Perceptron, a Multi-Layer Perceptron can effectively solve the XOR classification problem.

Discussion

This experiment demonstrated the evolution from the Single-Layer Perceptron (SLP) to the Multi-Layer Perceptron (MLP) through the XOR classification problem. Initially, the architecture and working of the Single-Layer Perceptron were studied, followed by a mathematical proof showing that the XOR dataset is not linearly separable. The XOR data points and the failed linear decision boundary further confirmed that a single straight line cannot correctly classify all four input combinations.

To overcome this limitation, a Multi-Layer Perceptron with one hidden layer was introduced. The hidden layer, together with the sigmoid activation function, enabled the network to learn nonlinear decision boundaries by transforming the input data into a new feature space. The TensorFlow implementation successfully classified all XOR input combinations, achieving **100% prediction accuracy** after training. The training loss also decreased steadily over the epochs, indicating that the model converged effectively and learned the XOR function without instability.

Overall, the experiment clearly demonstrated the importance of hidden layers in neural networks. It showed that while Single-Layer Perceptrons are suitable only for linearly separable problems, Multi-Layer Perceptrons can successfully solve nonlinear classification problems such as XOR. This experiment provides a strong foundation for understanding more advanced deep learning models and neural network architectures.

Conclusion

This experiment successfully demonstrated the limitations of the Single-Layer Perceptron and the advantages of the Multi-Layer Perceptron in solving nonlinear classification problems. The mathematical proof, data visualization, and TensorFlow implementation collectively showed that the XOR problem cannot be solved using a single linear decision boundary. By introducing a hidden layer and nonlinear activation functions, the Multi-Layer Perceptron accurately learned the XOR relationship and achieved **100% prediction accuracy**. The experiment highlights the significance of hidden layers in deep learning and establishes the Multi-Layer Perceptron as an effective model for solving complex classification tasks.